



What is Node.js

Node.js is a program that lets you **run JavaScript on your computer**, not just in a browser.

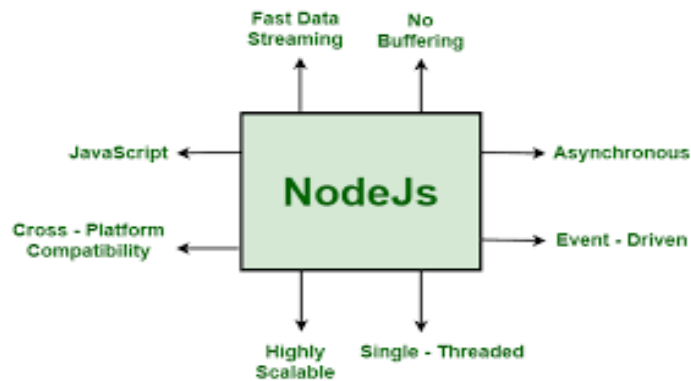
Normally, JavaScript runs inside a browser (like Chrome or Firefox), but with Node.js, you can use JavaScript to build things like:

- **Web servers** (that handle websites)
- **APIs** (that connect apps together)
- **Command-line tools** and even **real-time apps** (like chat apps)

Install Nodejs from here: <https://nodejs.org/>

Key Points

- **Free and open source** => anyone can use and modify it, runs on Windows, Mac, Linux, and more..
- **Fast** => built on Google Chrome's V8 JavaScript engine (the same one Chrome uses).
- **Efficient** => uses an "event-driven, non-blocking" system, meaning it can handle many tasks at the same time without slowing down.
- **Comes with built-in modules** => these are ready-made tools that make coding easier (like working with files, servers, or databases).



How to Run Node.js Code

To check node version:

node -v

Create a file named app.js

Save your code in a file, for example **app.js**, then run it in your terminal or command prompt with:

node app.js

What is npm?

npm is the package manager for Node.js.

It helps you install and manage third-party packages (libraries) to add more features to your apps.

💬 *npm is like the app store for Node but everything's free and open-source.*

How to check npm version

npm -v

Example:

```
npm install express
```

Node.js Global Objects

Node.js global objects are global in nature and available in all modules. You don't need to include these objects in your application; rather they can be used directly. These objects are modules, functions, strings and object etc. Some of these objects aren't actually in the global scope but in the module scope.

A list of Node.js global objects are given below:

- `__dirname`
- `__filename`
- `Console`
- `Process`
- `Buffer`
- `setImmediate(callback[, arg][, ...])`
- `setInterval(callback, delay[, arg][, ...])`
- `setTimeout(callback, delay[, arg][, ...])`
- `clearImmediate(immediateObject)`
- `clearInterval(intervalObject)`
- `clearTimeout(timeoutObject)`

Node.js `__dirname`

It is a string. It specifies the name of the directory that currently contains the code.

File: global-example1.js

```
console.log(__dirname);
```

Node.js `__filename`

It specifies the filename of the code being executed. This is the resolved absolute path of this code file. The value inside a module is the path to that module file.

File: global-example2.js

```
console.log(__filename);
```

JavaScript Modules

1. What Are Modules?

- **Modules** are reusable blocks of code that can be **imported and exported** between files.
- They help organize JavaScript code into smaller, maintainable pieces instead of one large file.
- Each module has its own **scope**, meaning variables and functions inside one module are **not automatically visible** to others.

2. Why Use Modules?

- ☺ Makes code **easier to maintain and debug**
- ☺ Encourages **code reusability**
- ☺ Prevents **naming conflicts**
- ☺ Supports **lazy loading** (load modules only when needed)

3. Types of Modules

- ✓ There are mainly two types of module systems in JavaScript:

ES MODULES
COMMON JS

Type	Used In	Syntax	Example
ES Modules (ESM)	Modern browsers, Node.js (latest versions)	import / export	export const x = 10;

CommonJS

Node.js (older system)

require /
module.exports

const x = require('./file');

4. Exporting in ES Modules

Exports allow parts of a module (variables, functions, classes) to be made available to other files.

a) Named Export

You can export multiple values from a module.

Example:

```
// file: math.js
export const add = (a, b) => a + b;
export const subtract = (a, b) => a - b;
```

Importing Named Exports:

```
// file: main.js
import { add, subtract } from './math.js';

console.log(add(2, 3)); // 5
```

You can also **rename** during import:

```
import { add as addition } from './math.js';
```

b) Default Export

Each module can have **only one default export**.

Example:

```
// file: logger.js
export default function log(message) {
  console.log(message);
}
```

Importing Default Export:

```
// file: app.js
import log from './logger.js';
log('Hello Modules!');
```

5. Combining Default and Named Exports

You can mix both in one module:

```
// file: utils.js
export const greet = name => `Hello, ${name}`;
export default function sayBye(name) {
  return `Goodbye, ${name}`;
}
```

Import:

```
import sayBye, { greet } from './utils.js';
```

6. Exporting/Importing All

Export everything from another file:

```
export * from './math.js';
```

Import everything as an object:

```
import * as math from './math.js';  
console.log(math.add(2, 2));
```

7. Dynamic Imports

Load modules **only when needed** (useful for performance optimization).

Example:

```
async function loadMath() {  
  const math = await import('./math.js');  
  console.log(math.add(3, 4));  
}  
loadMath();
```

8. Using Modules in HTML

To use ES Modules in the browser:

```
<script type="module" src="main.js"></script>
```

Modules are loaded **deferred** by default (executed after HTML is parsed).

9. CommonJS Syntax (Used in Node.js)

Export:

```
// file: math.js
module.exports = {
  add: (a, b) => a + b,
};
```

Import:

```
// file: main.js
const math = require('./math');
console.log(math.add(1, 2));
```

10. Key Differences: ES Modules vs CommonJS

Feature	ES Modules (ESM)	CommonJS
Syntax	import / export	require / module.exports
Asynchronous Loading	✔ Yes	✗ No
Used In	Browser + Node (modern)	Node.js (legacy)
Exports	Static	Dynamic
File Extension	.js, mjs (with "type": "module" in package.json)	.cjs or js

11. Practical Use Cases

- Splitting code by functionality (e.g., auth.js, db.js, utils.js)
- Creating reusable utility libraries
- Organizing frontend JavaScript for larger projects

- Lazy loading for faster app startup

Example

math.js

```
export function square(x) {  
  return x * x;  
}
```

utils.js

```
export function greet(name) {  
  return `Hello, ${name}`;  
}
```

main.js

```
import { square } from './math.js';  
import { greet } from './utils.js';  
  
console.log(greet('Shanice'));  
console.log(square(4));
```